

Scalable Notification System Implementation Guide

System Overview

This notification system is designed as a scalable, reliable platform for delivering notifications through multiple channels (email, SMS, push) with features such as deduplication, rate limiting, and retry handling. The system follows a microservices architecture where each service has a specific responsibility.

Architecture

The system consists of the following components:

Core Services

- Notification Orchestrator Service:** The entry point for the system. Receives notification requests, validates them, checks user preferences, and publishes messages to the appropriate RabbitMQ queues.
- Email Service:** Consumes email notification messages from RabbitMQ and sends emails using various providers (SendGrid, SMTP, AWS SES).
- SMS Service:** Consumes SMS notification messages from RabbitMQ and sends SMS messages using various providers (Twilio, AWS SNS, Nexmo).
- Push Service:** Consumes push notification messages from RabbitMQ and sends push notifications using various providers (FCM, APN, Web Push).
- Preferences Service:** Manages user notification preferences, including channels, categories, and delivery settings.
- Dashboard Service:** Provides APIs for monitoring and analytics of the notification system.

Infrastructure

- **RabbitMQ:** Message broker for asynchronous communication between services.
- **Redis:** Used for deduplication and rate limiting.
- **PostgreSQL:** Persistent storage for notification logs, user preferences, and analytics data.
- **Docker:** Containerization for easy deployment and scaling.
- **Docker Compose:** Orchestration for local development and testing.

Message Flow

1. A client application sends a notification request to the Notification Orchestrator Service.
2. The Orchestrator checks if the request is a duplicate using Redis.
3. The Orchestrator checks user preferences using the Preferences Service.

4. The Orchestrator applies rate limiting using Redis.
5. The Orchestrator publishes messages to the appropriate RabbitMQ queues.
6. The channel-specific services (Email, SMS, Push) consume the messages and send notifications.
7. The services log the delivery status to PostgreSQL.
8. The Dashboard Service provides APIs for monitoring and analytics.

Key Features

Asynchronous Processing

The system uses RabbitMQ as a message broker to decouple the request handling from the actual notification delivery. This allows the system to:

- Handle high traffic by queuing requests
- Process notifications in the background
- Scale horizontally by adding more workers
- Continue functioning even when some services are temporarily unavailable

Retry Mechanism

Each delivery service implements a retry mechanism with exponential backoff for failed notifications:

1. When a notification fails, it's requeued with a delay
2. The retry count is stored in the message headers
3. The delay increases exponentially with each retry
4. After reaching the maximum retry count, the message is sent to a Dead Letter Queue (DLQ)

Deduplication

The Notification Orchestrator uses Redis to prevent duplicate notifications:

1. Each notification request can include an idempotency key
2. The system checks if this key already exists in Redis
3. If it exists, the request is considered a duplicate and is skipped
4. If it's a new request, the key is stored in Redis with a TTL

Rate Limiting

Rate limiting is implemented to prevent spamming users:

1. Redis is used to track the number of notifications sent to each user
2. Rate limits can be configured per user and per channel
3. The count is tracked in a sliding window

4. Notifications exceeding the rate limit are rejected

User Preferences

The Preferences Service allows users to control their notification settings:

1. Enable/disable specific notification channels
2. Set quiet hours for each channel
3. Manage category preferences
4. Configure frequency limits
5. Set timezone information

Comprehensive Logging

All notification events are logged for auditing and analytics:

1. Delivery attempts
2. Delivery status
3. Error messages
4. Retry information
5. Provider details

Implementation Details

Notification Orchestrator Service

The Notification Orchestrator Service is the entry point for the system and handles the following responsibilities:

1. **Request Validation:** Validates incoming notification requests, ensuring all required fields are present and properly formatted.
2. **Deduplication:** Uses Redis to prevent duplicate notifications by checking idempotency keys.
3. **Rate Limiting:** Applies rate limiting rules using Redis to prevent notification spam.
4. **Preference Checking:** Calls the Preferences Service to check if the user has opted in for the notification.
5. **Message Publishing:** Publishes messages to the appropriate RabbitMQ queues based on the notification type.

Key components:

- `NotificationService`: Main service for handling notification requests
- `DeduplicationService`: Service for preventing duplicate notifications

- `RateLimitService`: Service for rate limiting notifications
- `RabbitMQClient`: Client for publishing messages to RabbitMQ
- `RedisClient`: Client for interacting with Redis

Email Service

The Email Service is responsible for sending email notifications:

1. **Message Consumption**: Consumes email notification messages from RabbitMQ.
2. **Email Delivery**: Sends emails using various providers (SendGrid, SMTP, AWS SES).
3. **Retry Handling**: Implements retry logic with exponential backoff for failed deliveries.
4. **Delivery Logging**: Logs delivery status and relevant information to PostgreSQL.

Supported email providers:

- SendGrid: For high-volume transactional emails
- SMTP: For generic email delivery
- AWS SES: For Amazon SES integration

SMS Service

The SMS Service handles SMS message delivery:

1. **Message Consumption**: Consumes SMS notification messages from RabbitMQ.
2. **SMS Delivery**: Sends SMS messages using various providers (Twilio, AWS SNS, Nexmo).
3. **Retry Handling**: Implements retry logic for failed deliveries.
4. **Delivery Logging**: Logs delivery status and relevant information to PostgreSQL.

Supported SMS providers:

- Twilio: For global SMS delivery
- AWS SNS: For Amazon SNS integration
- Nexmo/Vonage: For alternative SMS delivery

Push Service

The Push Service handles mobile and web push notifications:

1. **Message Consumption**: Consumes push notification messages from RabbitMQ.
2. **Push Delivery**: Sends push notifications using various providers (FCM, APN, Web Push).
3. **Retry Handling**: Implements retry logic for failed deliveries.
4. **Delivery Logging**: Logs delivery status and relevant information to PostgreSQL.

Supported push providers:

- Firebase Cloud Messaging (FCM): For Android and iOS
- Apple Push Notification Service (APN): For iOS-specific notifications
- Web Push: For browser notifications

Preferences Service

The Preferences Service manages user notification preferences:

1. **Preference Management:** Stores and retrieves user notification preferences.
2. **Channel Settings:** Manages per-channel settings (enable/disable, quiet hours).
3. **Category Settings:** Manages per-category settings (enable/disable, channel selection).
4. **Preference Checking:** Provides APIs for checking if a notification is allowed.

Preference types:

- Channel preferences: Email, SMS, Push
- Category preferences: Account, Marketing, Transactions, Updates, Alerts, News
- Global settings: Do Not Disturb, timezone

Dashboard Service

The Dashboard Service provides monitoring and analytics for the notification system:

1. **Log Retrieval:** Retrieves notification logs from PostgreSQL.
2. **Statistics:** Computes delivery statistics (success rates, volumes).
3. **Historical Data:** Provides historical data for analysis.
4. **Detail Views:** Shows detailed information about specific notifications.

API endpoints:

- `/api/v1/logs`: Get notification logs with pagination and filtering
- `/api/v1/logs/{notification_id}`: Get detailed information about a specific notification
- `/api/v1/stats`: Get notification statistics for a time range

Database Schema

The system uses PostgreSQL for persistent storage with the following main tables:

Email Delivery Logs (`email_delivery_logs`)

- `id`: Primary key (UUID)
- `notification_id`: Notification ID (for correlation)

- `user_id`: User ID (email address)
- `provider`: Email provider used (e.g., "sendgrid", "smtp", "ses")
- `status`: Delivery status (e.g., "pending", "sent", "failed")
- `subject`: Email subject
- `sent_at`: When the email was sent
- `delivered_at`: When the email was delivered
- `provider_message_id`: Message ID from the provider
- `error_message`: Error message if failed
- `retry_count`: Number of retry attempts
- `metadata`: Additional metadata (JSON)
- `created_at`: Record creation timestamp
- `updated_at`: Record update timestamp

SMS Delivery Logs (`sms_delivery_logs`)

- `id`: Primary key (UUID)
- `notification_id`: Notification ID (for correlation)
- `user_id`: User ID (phone number)
- `provider`: SMS provider used (e.g., "twilio", "sns", "nexmo")
- `status`: Delivery status (e.g., "pending", "sent", "failed")
- `body`: SMS body text
- `sent_at`: When the SMS was sent
- `delivered_at`: When the SMS was delivered
- `provider_message_id`: Message ID from the provider
- `error_message`: Error message if failed
- `retry_count`: Number of retry attempts
- `metadata`: Additional metadata (JSON)
- `created_at`: Record creation timestamp
- `updated_at`: Record update timestamp

Push Delivery Logs (`push_delivery_logs`)

- `id`: Primary key (UUID)
- `notification_id`: Notification ID (for correlation)
- `user_id`: User ID (device token)

- `provider`: Push provider used (e.g., "fcm", "apn", "webpush")
- `status`: Delivery status (e.g., "pending", "sent", "failed")
- `title`: Push notification title
- `body`: Push notification body
- `sent_at`: When the push notification was sent
- `delivered_at`: When the push notification was delivered
- `provider_message_id`: Message ID from the provider
- `error_message`: Error message if failed
- `retry_count`: Number of retry attempts
- `metadata`: Additional metadata (JSON)
- `created_at`: Record creation timestamp
- `updated_at`: Record update timestamp

User Preferences (`user_preferences`)

- `user_id`: Primary key (user identifier)
- `channels`: Channel preferences (JSON)
- `categories`: Category preferences (JSON)
- `do_not_disturb`: Global do not disturb flag
- `timezone`: User's timezone
- `created_at`: Record creation timestamp
- `updated_at`: Record update timestamp

Daily Statistics (`notification_daily_stats`)

- `date`: Primary key (date)
- `total`: Total notifications sent
- `email`: Email notifications sent
- `sms`: SMS notifications sent
- `push`: Push notifications sent
- `sent`: Notifications marked as sent
- `delivered`: Notifications marked as delivered
- `failed`: Notifications marked as failed
- `created_at`: Record creation timestamp
- `updated_at`: Record update timestamp

RabbitMQ Configuration

The system uses RabbitMQ as the message broker with the following configuration:

Exchanges

- `notifications`: Main topic exchange for notification messages
- `notifications.dlx`: Dead Letter Exchange for failed messages

Queues

- `email_notifications`: Queue for email notifications
- `sms_notifications`: Queue for SMS notifications
- `push_notifications`: Queue for push notifications
- `email_notifications.dlq`: Dead Letter Queue for failed email notifications
- `sms_notifications.dlq`: Dead Letter Queue for failed SMS notifications
- `push_notifications.dlq`: Dead Letter Queue for failed push notifications

Routing Keys

- `notification.email`: Routing key for email notifications
- `notification.sms`: Routing key for SMS notifications
- `notification.push`: Routing key for push notifications
- `notification.email.failed`: Routing key for failed email notifications
- `notification.sms.failed`: Routing key for failed SMS notifications
- `notification.push.failed`: Routing key for failed push notifications

Message Properties

- `content_type`: "application/json"
- `delivery_mode`: Persistent (2)
- `headers`: Contains retry information and other metadata
- `priority`: Message priority (0-9) based on notification priority

Redis Usage

Redis is used for various purposes in the system:

Deduplication

- Key Pattern: `dedup:{idempotency_key}`
- Value: Notification ID

- TTL: Configurable (default: 1 hour)
- Purpose: Prevent duplicate notifications by storing idempotency keys

Rate Limiting

- Key Pattern: `rate_limit:{user_id}:{notification_type}:{minute_window}`
- Value: Counter (number of notifications in the window)
- TTL: 2 minutes (to cover current and next minute)
- Purpose: Enforce rate limits by tracking notification counts in a sliding window

Caching

- Key Pattern: `prefs:{user_id}`
- Value: User preferences (JSON)
- TTL: Configurable (default: 5 minutes)
- Purpose: Cache user preferences to reduce database load

API Endpoints

Notification Orchestrator Service

Send Notification

- Endpoint: `POST /api/v1/notifications`
- Description: Send a notification to a user
- Request Body:

json

```
{
  "user_id": "user@example.com",
  "types": ["email"],
  "content": {
    "email": {
      "subject": "Welcome to Our Service",
      "body_text": "Thank you for signing up!",
      "body_html": "<p>Thank you for signing up!</p>"
    }
  },
  "priority": "medium",
  "idempotency_key": "unique-id-123"
}
```

- Response:

json

```
{
  "notification_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "sent",
  "created_at": "2023-04-01T12:34:56Z",
  "idempotency_key": "unique-id-123"
}
```

Send Bulk Notifications

- Endpoint: `POST /api/v1/notifications/bulk`
- Description: Send multiple notifications in a single request
- Request Body:

json

```
{
  "notifications": [
    {
      "user_id": "user1@example.com",
      "types": ["email"],
      "content": {
        "email": {
          "subject": "Welcome to Our Service",
          "body_text": "Thank you for signing up!"
        }
      }
    },
    {
      "user_id": "user2@example.com",
      "types": ["sms"],
      "content": {
        "sms": {
          "body": "Your verification code is 1234"
        }
      }
    }
  ],
  "batch_id": "batch-123"
}
```

- Response:

json

```
{
  "batch_id": "batch-123",
  "total": 2,
  "successful": 2,
  "failed": 0,
  "notifications": [
    {
      "notification_id": "550e8400-e29b-41d4-a716-446655440001",
      "status": "sent",
      "created_at": "2023-04-01T12:34:56Z"
    },
    {
      "notification_id": "550e8400-e29b-41d4-a716-446655440002",
      "status": "sent",
      "created_at": "2023-04-01T12:34:57Z"
    }
  ]
}
```

Preferences Service

Get User Preferences

- Endpoint: `GET /api/v1/preferences/{user_id}`
- Description: Get user notification preferences
- Response:

json

```
{
  "user_id": "user@example.com",
  "preferences": {
    "channels": {
      "email": {
        "enabled": true,
        "quiet_hours_start": null,
        "quiet_hours_end": null,
        "frequency_limit": null,
        "urgent_override": true
      },
      "sms": {
        "enabled": true,
        "quiet_hours_start": "22:00:00",
        "quiet_hours_end": "08:00:00",
        "frequency_limit": 5,
        "urgent_override": true
      },
      "push": {
        "enabled": true,
        "quiet_hours_start": null,
        "quiet_hours_end": null,
        "frequency_limit": null,
        "urgent_override": true
      }
    },
    "categories": {
      "account": {
        "enabled": true,
        "channels": {
          "email": true,
          "sms": true,
          "push": true
        }
      },
      "marketing": {
        "enabled": true,
        "channels": {
          "email": true,
          "sms": false,
          "push": false
        }
      }
    },
    "do_not_disturb": false,
    "timezone": "UTC"
  }
}
```

```
  },  
  "message": "Preferences retrieved successfully"  
}
```

Update User Preferences

- Endpoint: `PUT /api/v1/preferences/{user_id}`
- Description: Update user notification preferences
- Request Body:

json

```
{  
  "channels": {  
    "sms": {  
      "enabled": false  
    }  
  },  
  "do_not_disturb": true  
}
```

- Response:

json

```
{
  "user_id": "user@example.com",
  "preferences": {
    "channels": {
      "email": {
        "enabled": true,
        "quiet_hours_start": null,
        "quiet_hours_end": null,
        "frequency_limit": null,
        "urgent_override": true
      },
      "sms": {
        "enabled": false,
        "quiet_hours_start": "22:00:00",
        "quiet_hours_end": "08:00:00",
        "frequency_limit": 5,
        "urgent_override": true
      },
      "push": {
        "enabled": true,
        "quiet_hours_start": null,
        "quiet_hours_end": null,
        "frequency_limit": null,
        "urgent_override": true
      }
    },
    "categories": {
      "account": {
        "enabled": true,
        "channels": {
          "email": true,
          "sms": true,
          "push": true
        }
      },
      "marketing": {
        "enabled": true,
        "channels": {
          "email": true,
          "sms": false,
          "push": false
        }
      }
    },
    "do_not_disturb": true,
    "timezone": "UTC"
  }
}
```

```
    },  
    "message": "Preferences updated successfully"  
  }  
}
```

Dashboard Service

Get Notification Logs

- Endpoint: `GET /api/v1/logs?page=1&page_size=20¬ification_type=email&status=sent&user_id=user@example.com`
- Description: Get notification logs with pagination and filtering
- Response:

```
json  
{  
  "total": 100,  
  "page": 1,  
  "page_size": 20,  
  "logs": [  
    {  
      "id": "550e8400-e29b-41d4-a716-446655440000",  
      "notification_id": "550e8400-e29b-41d4-a716-446655440000",  
      "user_id": "user@example.com",  
      "type": "email",  
      "status": "sent",  
      "provider": "sendgrid",  
      "subject": "Welcome to Our Service",  
      "sent_at": "2023-04-01T12:34:56Z",  
      "delivered_at": null,  
      "error_message": null,  
      "retry_count": 0,  
      "created_at": "2023-04-01T12:34:56Z",  
      "updated_at": "2023-04-01T12:34:56Z"  
    }  
  ]  
}
```

Get Notification Detail

- Endpoint: `GET /api/v1/logs/{notification_id}`
- Description: Get detailed information about a specific notification
- Response:

json

```
{
  "notification_id": "550e8400-e29b-41d4-a716-446655440000",
  "logs": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "notification_id": "550e8400-e29b-41d4-a716-446655440000",
      "user_id": "user@example.com",
      "type": "email",
      "status": "sent",
      "provider": "sendgrid",
      "subject": "Welcome to Our Service",
      "sent_at": "2023-04-01T12:34:56Z",
      "delivered_at": null,
      "error_message": null,
      "retry_count": 0,
      "created_at": "2023-04-01T12:34:56Z",
      "updated_at": "2023-04-01T12:34:56Z"
    }
  ]
}
```

Get Statistics

- Endpoint: `GET /api/v1/stats?time_range=last_7_days`
- Description: Get notification statistics for a time range
- Response:

json

```
{
  "time_range": "last_7_days",
  "start_date": "2023-03-26",
  "end_date": "2023-04-01",
  "stats": {
    "total": 1000,
    "sent": 950,
    "delivered": 900,
    "failed": 50,
    "pending": 0,
    "retrying": 0,
    "by_type": {
      "email": 600,
      "sms": 300,
      "push": 100
    },
    "by_status": {
      "pending": 0,
      "sent": 950,
      "delivered": 900,
      "failed": 50,
      "retrying": 0
    }
  },
  "daily_stats": [
    {
      "date": "2023-03-26",
      "total": 150,
      "email": 100,
      "sms": 40,
      "push": 10,
      "sent": 145,
      "delivered": 140,
      "failed": 5
    },
    {
      "date": "2023-03-27",
      "total": 140,
      "email": 90,
      "sms": 35,
      "push": 15,
      "sent": 135,
      "delivered": 130,
      "failed": 5
    }
  ]
}
```

```
]
}
```

Deployment

Docker Compose

The system can be deployed locally or in a development environment using Docker Compose:

```
bash
docker-compose up -d
```

This will start all the services, RabbitMQ, Redis, and PostgreSQL. The services will be accessible at the following URLs:

- Notification Orchestrator: <http://localhost:8000>
- Preferences Service: <http://localhost:8001>
- Dashboard Service: <http://localhost:8002>
- RabbitMQ Management: <http://localhost:15672> (user: admin, password: adminpass)

Kubernetes

For production deployment, the system can be deployed to Kubernetes using Helm charts. Each service should have its own deployment and service, and can be scaled independently based on load.

Key considerations for Kubernetes deployment:

- Use StatefulSets for RabbitMQ and Redis to maintain stable network identities
- Use Persistent Volume Claims for PostgreSQL to ensure data persistence
- Configure health checks and readiness probes for all services
- Set up autoscaling based on CPU and memory usage
- Use Kubernetes Secrets for storing sensitive information
- Configure ingress for external access to the services

Database Initialization

Before starting the services, initialize the database schema:

```
bash
python scripts/init_db.py
```

This will create all the required tables in the PostgreSQL database.

Scaling Considerations

The notification system is designed to be highly scalable:

Horizontal Scaling

- **Stateless Services:** All services are designed to be stateless, allowing for easy horizontal scaling.
- **Message Queue:** RabbitMQ decouples request handling from notification delivery, allowing each service to scale independently.
- **Independent Scaling:** Each service can be scaled based on its specific load. For example, the Email Service might need more instances than the SMS Service.

Database Scaling

- **Read Replicas:** PostgreSQL can be configured with read replicas to handle high read loads.
- **Sharding:** As the system grows, database sharding can be implemented to distribute the load.
- **Connection Pooling:** All services use connection pooling to efficiently manage database connections.

RabbitMQ Scaling

- **Clustering:** RabbitMQ can be deployed as a cluster for high availability and throughput.
- **Mirrored Queues:** Critical queues can be mirrored across nodes to prevent data loss.
- **Federated Exchanges:** For multi-region deployments, federated exchanges